

PENETRATION TESTING REPORT

Red Team Exercise Manager

Security Assessment

Client:	Academic Project Security Review
Application:	Red Team Exercise Manager – AI Microservice
URL:	http://127.0.0.1:5000
Assessment Period:	May 03-06, 2026
Report Date:	May 06, 2026
Prepared By:	Shreyanka B A
Lead Tester:	Security Analyst - Ethical Hacking Course
Classification:	CONFIDENTIAL

CONFIDENTIALITY NOTICE: This document contains sensitive security information and is intended solely for authorized personnel. Unauthorized disclosure, copying, or distribution of this document is strictly prohibited. This report must be stored securely and handled in accordance with your organization's data protection policies.

Table of Contents

1. Executive Summary	4
2. Scope and Methodology	5
2.1 Testing Scope	5
2.2 Testing Methodology	6
2.3 Tools and Technology	6
3 .Detailed Findings	7
3.1 Critical Severity Findings	7
3.2 high Severity Findings	7
3.3 Medium Severity Findings	8
3.4 Low Severity Findings	8
4. Technical Details	9
5. Validation Evidence	9
6. Impact Assessment	10
7. Remediation	11

1. Executive Summary

Assessment Overview:

A security assessment was conducted on the AI microservice component of the Red Team Exercise Manager project. The objective was to identify security weaknesses in the currently deployed application service.

Overall Risk Level: MEDIUM

Testing identified four vulnerabilities:

- 2 Medium severity

- 2 Low severity

No Critical or High severity vulnerabilities were detected.

Key Findings:

The following table summarizes the vulnerabilities discovered during this assessment:

Severity	Count	Percentage	CVSS Range	Priority
Critical	0	0	9.0 - 10.0	P0 - Immediate
High	0	0	7.0 - 8.9	P1 - Urgent
Medium	2	50%	4.0 - 6.9	P2 - Important
Low	2	50%	0.1 - 3.9	P3 - Standard
Total	4	100%	-	-

Critical Recommendations:

Based on the severity and exploitability of identified vulnerabilities, the following immediate actions are recommended:

- **Immediate (0-24 hours):** Address all Critical severity vulnerabilities, particularly SQL Injection and Command Injection flaws
- **Short-term (1-7 days):** Remediate High severity findings including XSS vulnerabilities and file upload controls
- **Medium-term (1-4 weeks):** Implement comprehensive security controls including input validation, output encoding, and security headers
- **Long-term (1-3 months):** Establish secure development lifecycle practices and conduct regular security assessments

2. Scope and Methodology

2.1 Testing Scope

In-Scope Components:

The security assessment covered the following components of the DVWA application:

The security assessment covered the following components of the Red Team Exercise Manager AI microservice:

- Web application root endpoint (/) accessibility and response behavior
- HTTP response header security configuration
- Server information disclosure through response headers
- API endpoint accessibility validation (/health, /ai/search)
- Input validation testing using custom Python scripts
- Service availability and response reliability checks
- Automated web vulnerability scanning using OWASP ZAP

Out-of-Scope:

The following items were explicitly excluded from this assessment:

- Docker/container security testing
- Denial of Service (DoS) testing
- Network infrastructure testing
- Social engineering attacks
- Physical security controls

2.2 Testing Methodology

The assessment followed security review practices based on the OWASP Testing Guide and included the following phases:

Phase	Activities	Duration
1. Setup & Deployment	• Repository cloning • Virtual environment setup • API key configuratio • Local service execution	2 hours
2.Reconnaissance	• Information gathering • Technology identification • Route discovery	1 hour
3. Automated Scanning	• Vulnerability scanning using OWASP ZAP • HTTP response analysis	2 hours
4. Manual Validation	• Custom Python testing scripts • Endpoint accessibility checks • Header validation	2 hours
5.Reporting	• Documentation • Screenshot collection • Remediatio recommendations	1 hours

2.3 Tools and Technologies

The following tools and technologies were utilized during this assessment:

- **OWASP ZAP** – Automated web vulnerability scanning and HTTP security header analysis
- **Python 3.x** – Custom security validation scripts for endpoint and header testing
- **curl** – Manual API request and endpoint validation
- **Visual Studio Code** Terminal – Local command execution and testing
- **Firefox** Browser – Service accessibility testing and response verification
- **Kali Linux** – Primary security testing platform

- **Flask** – AI microservice framework under assessment

3.Detailed Findings

3.1 Critical Severity Findings : No Critical Severity vulnerabilities were identified during this assessment.

3.2 high Severity Findings : No High Severity vulnerabilities were identified during this assessment.

3.3 Medium Severity Findings

Finding #1: Content Security Policy Header Not Set

Severity: MEDIUM

CVSS v3.1 Score: 5.4 (Medium)

CWE: CWE-693: Protection Mechanism Failure

Affected URL: <http://127.0.0.1:5000/>

Discovery Date: May 05, 2026

Description :The application response does not include a Content Security Policy (CSP) header.Absence of CSP reduces browser-side restrictions on loading scripts and other resources, which may increase exposure to client-side attacks such as cross-site scripting.

Finding #2: Missing Anti-clickjacking Header

Severity: MEDIUM

CVSS v3.1 Score: 4.8 (Medium)

CWE: **CWE-1021:** Improper Restriction of Rendered UI Layers

Affected URL: <http://127.0.0.1:5000/>
Discovery Date: May 05, 2026

Description : The application does not include anti-clickjacking protection. Missing X-Frame-Options header allows application pages to potentially be embedded in malicious frames.

3.4 Low Severity Findings

Finding #3: Server Version Information Disclosure

Severity: LOW

CVSS v3.1 Score: 3.1 (Low)

CWE: **CWE-200:** Information Exposure

Affected URL: <http://127.0.0.1:5000/>

Discovery Date: May 05, 2026

Description : Server response discloses framework and runtime version information

Finding #4: Missing X-Content-Type-Options Header

Severity: LOW

CVSS v3.1 Score: 2.6 (Low)

CWE: **CWE-16:** Configuration

Affected URL: <http://127.0.0.1:5000/>

Discovery Date: May 05, 2026

Description : The application does not include MIME-sniffing protection

4. Technical Details

The application response headers were inspected through automated scanning and manual Python-based validation.

Observed missing security headers:

- Content-Security-Policy
- X-Frame-Options
- X-Content-Type-Options

Observed disclosed server header:

- Werkzeug/3.1.8 Python/3.13.9

5. Validation Evidence

The following tests confirmed the findings:

Test 1: Root Endpoint Accessibility

Request:

GET /

Result:

HTTP 200 OK

Test 2: Security Header Inspection

Method: Python custom script + OWASP ZAP

Observed Missing Headers:

Content-Security-Policy

X-Frame-Options

X-Content-Type-Options

Test 3: Server Information Disclosure

Observed Response Header:

Server: Werkzeug/3.1.8 Python/3.13.9

Result:

Framework and runtime versions exposed.

Test 4: Endpoint Validation

Requests Tested:

GET /health

POST /ai/search

Result:

404 Not Found

Observation:

API documentation and implemented routes appear inconsistent.

6. Impact Assessment:

Confidentiality Impact: LOW No direct data exposure or credential leakage was identified during testing. However, server version disclosure may provide attackers with technology fingerprinting information useful for reconnaissance.

Integrity Impact: LOW - No vulnerabilities were identified that would allow unauthorized modification of application data or system configuration.

Availability Impact: LOW - No issues affecting service availability or application stability were identified during testing.

Business Impact:

- Reduced HTTP security hardening posture
- Increased exposure to browser-based attacks such as clickjacking
- Technology stack disclosure through server headers
- Minor documentation inconsistency in exposed API routes

7. Remediation:

Immediate Actions (Priority P2 – Important):

- Implement missing Content Security Policy header
- Add anti-clickjacking protection using X-Frame-Options

- Add MIME-sniffing protection with X-Content-Type-Options
- Suppress framework and Python version disclosure in server headers

Additional Security Controls:

- Implement secure HTTP response headers across all application responses
- Disable unnecessary server version disclosure in production environments
- Standardize API route documentation and endpoint management
- Configure secure deployment settings for production Flask services
- Perform regular automated vulnerability scanning using **OWASP ZAP**
- Integrate security validation checks into development workflow
- Conduct periodic manual security reviews before deployment

Verification: After implementing fixes, perform regression testing with the same payloads to ensure vulnerability is completely remediated. Validate that error messages do not reveal database structure.

